Project 7

Project Name:

Web-based Supply Chain Management system for a company (WBSCM)

Team Members:

- Arnib Alam Farooqui
- Daniel Hernandez

Final State of System Statement:

The semester project aimed to design and implement a web-based Supply Chain Management (SCM) System using Java and MySQL. The system was intended to streamline and automate the entire supply chain lifecycle within a company. Key objectives included easy addition and management of suppliers, comprehensive product details display, efficient addition of new products, and a smooth purchase process, individual customer cart management, and an order tracking mechanism.

Outcome:

The system, upon completion, is equipped to handle various supply chain operations, featuring efficient supplier Product management, detailed product displays, seamless addition of new products, and a smooth purchasing procedure based on Customer present in the database. The system prioritized peak performance and scalability.

Project Requirements and Responsibilities:

The project focused on designing, developing, and implementing a robust SCM system using Java and MySQL. Key functionalities included Adding Products by Suppliers, Viewing Product Details, Adding Products, and Purchasing Products, Order status tracking.

Functional Capabilities & Responsibilities:

Adding Products by Suppliers:

- *Requirement:* Allow users to seamlessly add new products to the system.
- *Responsibility:* Create a user-friendly interface for adding new product details, securely storing them in the MySQL database.

Viewing Product Details:

- *Requirement:* Enable users to view comprehensive product descriptions.
- *Responsibility:* Create an interface fetching and displaying detailed product information from the MySQL database.

Purchasing Products:

- *Requirement:* Enable users to conveniently purchase products.
- *Responsibility:* Develop a purchase feature guiding users through the checkout process, ensuring real-time updates in the MySQL database post-purchase.

Individual Customer Cart Management:

- Requirement: Facilitate efficient management of individual customer shopping carts.
- Responsibility: Implement features allowing customers to add, and modify items in their respective shopping carts. Ensure real-time updates and synchronization with the MySQL database to reflect accurate cart contents in the order table.

Order Tracking System:

- Requirement: Establish a system to track the status of orders.
- Responsibility: Develop a comprehensive order tracking system, enabling users/observers to monitor the entire lifecycle of their orders. Implement functionalities to track order status changes, shipment details, and delivery updates. Integrate this system seamlessly with the MySQL database for real-time and accurate information.

Integration of Extended Warranties:

- Requirement: Provide additional warranty benefits for specific product types.
- Responsibility: Implement a feature that automatically applies and displays extended warranty information based on the type of product purchased. Develop a system that calculates and communicates warranty details for electronic items (2 years) and furniture (5 years). Ensure accurate representation in the user interface and synchronization with the MySQL database for warranty-related information.

Assignment of Product IDs:

- Requirement: Assign unique ProductIDs to each product for effective tracking.
- Responsibility: Develop a system that assigns and manages unique ProductIDs for every product added to the system by suppliers. Ensure that each ProductID is securely stored in the MySQL database, establishing a reliable linkage between products and their identifiers.

Customer Information Management:

Requirement: Maintain comprehensive customer information for subscribers.

- Responsibility: Storing and managing customer details, including user ID, password, email, customer name, address, registration date, and preferred payment mode. Ensure data integrity and security in the MySQL database, providing seamless access to customer information when required.

Checkout Process:

- Requirement: Facilitate a secure and efficient checkout process for customers.
- Responsibility: Develop a streamlined checkout feature that guides users through the payment process, ensuring secure transactions based on their preferred payment mode. Integrate the checkout system with the MySQL database to record and update order details, providing a seamless experience for users completing their purchases.

Design Process:

The application collaborates with two categories of suppliers specializing in electronic and furniture products, providing essential product details such as name, description, price, and weight. As an added benefit for customers using our platform, we offer extended warranty periods, including a 2-year warranty for electronic items and a fixed 5-year warranty for furniture. Our application efficiently manages product tracking by assigning unique ProductIDs to each item.

In response to the growing number of subscribers, our application maintains comprehensive customer information, including user ID, password, email, customer name, address, registration date, and preferred payment mode. Customers engage in shopping by adding items to their carts. The checkout process ensures secure payment based on the preferred mode, and the order is successfully placed. The order_item table in the database keeps a record of purchased items, while the application's in-house database (order_table) monitors order status and shipping dates. Notifications are triggered when the order status transitions from 'pending' to 'shipped.'

Features Not Implemented and Reasons:

Intensive Front-End Development:

Reason: We have changed the intensive web based UI interface to Simple Text-based UI as suggested in our Proj-5 feedback. This directed our attention towards enhancing the backend functionalities, as a significant portion of our efforts centered on backend development and the integration of OOAD patterns within our code. We produced considerable volumes of code that aligned closely with our class diagram.

Additional In-Depth Details such as Stock Management, user management, Report publishing:

Reason: While the basic features, such as cart implementation, profile details, order management have been successfully implemented, the team faced constraints in integrating more in-depth details such as categories for products, stock management, and user login authentication and user-related database management. We also did not implement Report class This limitation may be attributed to the prioritization of demonstrating solid back-end functionality, architecture, and design patterns.

Data Storage:

Data persisted using MySQL. Version: 8.0.34

Changes from Project 5 and 6:

Changes in Scope: Owing to time limitations, we had to revise our previous ambition to develop a full-stack web application as outlined in Project 5. Instead, our focus shifted to building and testing a functional prototype, providing a sturdy foundation for future expansions.

Progress Made: Aligned with our plan from Project 6, we advanced in developing the back-end and initiated basic works on front-end implementation. The database connections to the back-end also saw significant progress.

Contrasting Project 5 to our current reality, we've shifted our approach towards creating a Spring application that provides smooth access to the MySQL Database. By leveraging Spring Initializr, we've not only streamlined the setup process but also ensured that all necessary dependencies for our application are effectively included.

During the course of development, we did away with some unnecessary classes to maintain code cleanliness, and incorporated the Data Access Object Pattern to further enhance our design. As a result, we regularly updated our UML diagrams to keep them accurate to ongoing changes in system design.

For frontend development we use Rest controller

To test the functionality of our REST API calls, we employed Postman, an API development tool, enabling us to send, receive and thoroughly analyze HTTP(POST, GET or DELETE) data to and from the server. This hands-on testing strategy has ultimately bolstered our confidence in the robustness of our API.

Third-Party Code vs. Original Code Statement:

In the process of building our project, we aimed for a balance between leveraging third-party solutions for efficiency and writing original code to achieve our specific requirements.

Original Code: All backend and front-end code specific to the implementation of our Supply Chain Management system is original. This includes classes such as payment strategies, carts, customers, and all the logic associated with these entities. This code was developed from scratch based on the UML diagram.

The front-end code is written using the @RestController to handle the HTTP requests and responses in the Spring MVC controller class. This code is tailored to the specific needs of our use cases and provides our unique application logic.

Third-Party Code:

We utilized the following third-party tools, frameworks and libraries:

Spring Boot Framework - We used Spring Boot to simplify the creation of stand-alone, production-grade Spring-based applications. This tool was crucial for setting up our project swiftly and provided various useful defaults. Spring Boot

Spring Initializr - This quickstart generator assisted us in setting up our project, helping us pull in all the necessary dependencies for our application in an organized manner. Spring Initializr

MySQL - Our chosen database system. This was used to create and interact with our application's database. MySQL

Spring MVC - The @RestController is a part of the Spring MVC framework, a widely used third-party framework for creating web applications on the Java platform. The use of Spring's @RestController helped in managing and routing incoming HTTP requests to the appropriate controller methods and then rendering the HTTP responses for those requests. Spring MVC

Postman - This API development tool was used for testing REST API calls, sending and receiving HTTP (POST, GET or DELETE) requests to the server. Postman

While using these third-party tools, frameworks and libraries, we also referred to multiple online tutorials and examples to understand their use better.

UML Class Diagram:

Design Patterns Usage:

1. Abstract Factory and Factory Pattern with Products creation & Product type

The Abstract Factory Pattern provides an interface for creating families of related objects without specifying their concrete classes. This pattern can be applied to a product-creation mechanism in our system. AbstractProductFactory interface can define a method createProduct(), and concrete factories like ElectronicProductFactory and FurnitureProductFactory will create and return an instance of ElectronicProduct and FurnitureProduct respectively. This way, the system can produce different types of products through a common interface, which improves the modularity and flexibility of the code.

The Factory Pattern has been applied in our SCM system for creating different types of Product objects. This pattern provides an interface for creating an object in a superclass, but it allows subclasses to alter the type of an object that will be created. For instance, our Product hierarchy may include classes like ElectronicsProduct, FurnitureProduct, etc., each representing a unique type of product. Depending upon the type specified during the invocation of createProduct(), the factory generates and returns an instance of the corresponding product type.

2. Observer Pattern with Order tracking

The Observer Pattern defines a one-to-many dependency between objects, so when one object changes state, all its dependents are notified automatically. This is useful in our system where we have Order objects whose state may change over time. This pattern creates a one-to-many dependency between objects, so that when one object (the status of the Order for example: OrderID, Order status, Shipment date)changes its state, all its dependents (like the Customer or UI) are notified automatically and can take appropriate actions.

3. Strategy Pattern with Payment

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. This is a perfect fit in our system for different payment methods. PaymentStrategy is an interface that declares a method executePayment(). This interface is implemented by multiple concrete strategy classes like CreditCardStrategy and PaypalStrategy, each providing a different payment method. An Order object maintains a reference to a PaymentStrategy object and can execute a payment using the current payment strategy, providing flexibility to switch payment methods at runtime.

4. Singleton Pattern with Database connection

In our SCM system, the Singleton Pattern has been used for managing the database connection. The pattern ensures that only a single instance of a database connection is created and shared across all parts of the application needing to access the database.
A DatabaseConnection class in our application encapsulates the database connection setup. This class exposes a static method, often named getInstance(), to provide a global access point to the instance. Once an instance of the DatabaseConnection class is created, the same instance is returned for all subsequent calls to getInstance().
This ensures that your application uses a single database connection throughout its lifecycle, therefore avoiding the overhead of setting up and tearing down multiple connections. Additionally, it promotes consistent and synchronized database access across different parts of our application, preventing possible data inconsistencies due to multiple connections.

5. Data Access Object (DOA) pattern with Customer, Product, Payment, Order

In our SCM system, the DAO Pattern is applied to entities such as Customer, Product, Payment, and Order. This pattern separates the data access logic or persistence mechanism from the business logic.
For each of these entities, a related DAO interface is created, e.g., CustomerDAO, ProductDAO, PaymentDAO, and OrderDAO. These interfaces declare the necessary CRUD (Create, Retrieve, Update, Delete) operations and other methods required for the entities. As a result of this separation, the rest of the application interacts with the DAO layer instead of directly with the database, serving decoupling and abstraction.

6. Model-View-Controller (MVC) Pattern with ApiController

The 'ApiController' class, as presented in the context of a RESTful API in a Spring Boot application, aligns with the principles of the Controller component in the MVC design pattern. In the MVC pattern:
Model: Represents the data and business logic of the application. In your case, this would include entities such as Product, Order, and associated services.
View: Represents the presentation layer responsible for displaying the user interface. In a RESTful API, the view is often abstracted away as the data is typically returned in a structured format (like JSON) and doesn't directly deal with rendering views.
Controller: Acts as an intermediary between the Model and the View. It receives user input, processes it (possibly invoking Model services), and returns the appropriate response. The ApiController class, in this context, handles incoming HTTP requests, delegates the processing to appropriate services, and returns HTTP responses.

Project 5: Link to UML Class Diagram

Project 7:

Throughout the course of Projects 5 to 7, there have been significant evolutions and enhancements in our SCM system. Many of these changes came as a result of insights gathered through the design and development phases, aiming to improve the system's functionality, flexibility, and maintainability.

1.Integration and Scalability Changes: Originally, the design in Project 5 aimed for full-stack development, including extensive front-end work. However, due to time constraints, we scaled back on full-stack development in Project 6. Instead, our focus shifted towards the back-end development integrating the system with the database while initiating basic front-end efforts. This ensured that the mission-critical parts of the system - the backend and database - were strong and fully functional, creating a scalable foundation for future enhancements to the user interface.

2. Database and Back-End Changes: In implementing the back-end and database, we transformed our initial design into a more efficient one. We adopted Spring application for seamless database access, replacing redundant classes, and introducing Data Access Object (DAO) patterns to improve data access. Incorporating Spring Initializr helped to pull in all necessary dependencies necessary for the application. This significantly enhanced our database connection management and streamlined the setup process.

3. New Design Pattern addition: A significant evolution in our application stemmed from the incorporation of various design patterns, including Factory and DAO patterns.

Factory Pattern: We chose to use the Factory Pattern in our application as a way to manage the creation of Product objects. This pattern provided a way to encapsulate the instantiation logic based on the type of product needed, which made our code easier to read and maintain. By introducing the Factory Pattern, we abstracted the process of object creation and kept our system flexible, allowing us to add or modify product types with minimal impact on the existing code base.

Data Access Object (DAO) Pattern: The DAO pattern was another critical incorporation in our application. It was chosen to isolate the application's business logic from the database access logic. This pattern has helped us to structure our code better by separating the concerns of data persistence and making it more modular. By segregating the data access into a separate DAO layer, we increased the adaptability and maintainability of our system. Notably, if there are changes in the data source or the way data is accessed, we can adjust this within the DAO layer without largely affecting the business logic. Thus, by using DAO, our application became less dependent on the persistence layer architecture, leading to easier updates and modifications to the data source or schema in the future.

4. Testing Changes: We also made strides in testing our system. The development of REST API calls and the implementation of DAO made it necessary to employ tools such as Postman for

sending, receiving, and analyzing HTTP data to the server. This offered valuable feedback for refining our application and confirmed the robustness of our API.

The culmination of these changes resulted in a functional, efficient, and scalable SCM system by Project 7.

Statement on the OOAD process for our Overall Semester Project:

Over the course of the semester, engaging in the Object-Oriented Analysis and Design (OOAD) process for our web-based Supply Chain Management system (WBSCM) has been enlightening. It offered us a structured approach to analyze, design, and implement our system using object-oriented technologies. The iterative nature of the process enabled us to enhance our design gradually and address complex aspects of the system effectively. We consistently applied OO principles and integrated various design patterns (like Factory, Abstract Factory, Observer, Singleton, Strategy, and Data Access Object) which significantly contributed to a more robust, adaptable, and maintainable system.

Three key design process elements/issues:

1. Integration of Diverse Components (Challenge): A key issue we faced was ensuring the smooth integration of various system components. It was an iterative process to make sure all parts, such as back-end logic, database integration, handling HTTP requests, and front-end

development, worked harmoniously. It required a clear understanding of the entire system and careful syncing between team members.

2. Application of Design Patterns (Positive): Using design patterns to address common problems was an integral part of our design process. Patterns like Factory, Observer, Singleton, Strategy, and Data Access Object helped us address different design problems in a standard, efficient, and reusable way. This not only improved the structure of our code but also made the system more flexible and easier to manage.

3. Iterative Enhancement (Positive): The OOAD process encourages an iterative development approach, which we found highly beneficial. This allowed us to incrementally add to our design and continually refine it. From Project 5 to Project 7, we underwent a transformative journey from designing raw UML diagrams and identifying patterns to implementing actual code and testing prototypes. The iterative nature of our work allowed room for learning, adaptations, and innovations.

Sources throughout development:

https://start.spring.io/

https://spring.io/projects/spring-boot

https://www.w3schools.com/

https://stackoverflow.com/

https://www.geeksforgeeks.org/how-to-create-a-rest-api-using-java-spring-boot/

https://www.linkedin.com/pulse/database-integration-postman-xmysql-umme-habiba?utm_source=share&utm_medium=member_ios&utm_campaign=share_via

https://support.postman.com/hc/en-us/articles/4406635188375-How-to-connect-Postman-to-a-database

https://support.postman.com/hc/en-us/articles/4406635188375-How-to-connect-Postman-to-a-database

https://support.postman.com/hc/en-us/articles/4406635188375-How-to-connect-Postman-to-a-database

https://www.youtube.com/watch?v=R_lWdLUheH4

References:

1. Kodali, K. C. (2019). Development of Web Based Application for Supply Chain Management. St. Cloud State Repository. Retrieved from https://repository.stcloudstate.edu/cgi/viewcontent.cgi?article=1070&context=mme_etds
2. Investopedia. (n.d.). Supply Chain Management (SCM). Retrieved from https://www.investopedia.com/terms/s/scm.asp
3. ResearchGate. (n.d.). Supply Chain Management. Retrieved from https://www.researchgate.net/publication/304194361_Supply_Chain_Management
4. CopyAssignment. (n.d.). Supply Chain Management System in Java. Retrieved from https://copyassignment.com/supply-chain-management-system-in-java/
5. GeeksforGeeks. (n.d.). Design a Logistics System. Retrieved from https://www.geeksforgeeks.org/design-a-logistics-system/
6. Oracle. (n.d.). MySQL: The world's most popular open-source database. Retrieved from https://www.mysql.com/
7. Pivotal Software. (n.d.) Spring Boot. Retrieved from https://spring.io/projects/spring-boot
8. https://www.tutorialspoint.com/design_pattern/index.htm
9. Postman. (n.d.) Postman | The Collaboration Platform for API Development. Retrieved from https://www.postman.com/

10. Martin, B. (2021). Designing a RESTful Web API. Retrieved from https://briancaffey.github.io/2017/11/10/designing-a-restful-api-with-django-and-drf.html
11. Beamon, B. (1998). Supply Chain Design and Analysis: Models and Methods. International Journal of Production Economics, V. 55, Issue 3, pp. 281-294. Link
12. Tsai, M., & Huang, H. (2003). An ordered logistic regression model for a classification problem in supply chain management. European Journal of Operational Research, 144(3), 543-554. Retrieved from https://www.sciencedirect.com/science/article/abs/pii/S0377221702008068
13. Fjose, S., & Jakobsen, E.W. (2011). Ch 3: Supply Chain Management in Construction. E-Business in Construction. IntechOpen. Retrieved from https://www.intechopen.com/chapters/17154